

Documento 07: Guía Pedagógica Paso a Paso para Aprender y Programar en PHP (SGDM - ASCEND)

1. Introducción y Filosofía de Aprendizaje

Esta guía está escrita para orientarte en tu proceso de aprendizaje de **PHP (Hypertext Preprocessor)** y la programación orientada a objetos en el backend.

Aquí responderemos a tu duda central: **¿Por dónde empiezo a programar? ¿Hago modelos con getters, setters y variables privadas como en Java, o sigo otro camino?**

Te explicaremos el orden lógico de construcción para que puedas ir programando módulo por módulo con total seguridad, entendiendo qué hace cada línea de código.

2. ¿Debo hacer Clases con Atributos, Getters y Setters como en Java?

En el mundo de la programación web con PHP existen dos formas de abordar los Modelos:

Enfoque A: POO Estilo Java clásico (Entidades con Getters y Setters)

Creas una clase con `$id`, `$email`, `$nombreCompleto` privados y 20 métodos `getId()`, `setId()`, `getEmail()`, `setEmail()`.

- **Ventaja:** Muy estricto y formal.
- **Desventaja en PHP para proyectos web:** Requiere escribir cientos de líneas repetitivas (*código boilerplate*) para simplemente pasar datos de MySQL al formulario HTML.

Enfoque B (Recomendado en la Industria y UTU): Patrón DAO / Data Mapper con Arreglos Asociativos Tipados

En PHP moderno con **PDO**, la base de datos ya devuelve los registros directamente como arreglos asociativos (`$usuario['email']`, `$usuario['nombre_completo']`).

- **Tu clase Modelo (Usuario.php)** no es un contenedor pasivo de variables, sino un **Objeto de Acceso a Datos (DAO)** que contiene los métodos de negocio (`obtenerTodos()`, `obtenerPorId($id)`, `registrar($datos)`, `actualizar($id, $datos)`, `cambiarEstado($id, $estado)`).
 - **Resultado:** Tu código es 4 veces más rápido de programar, mucho más legible y perfectamente alineado con los estándares de frameworks como Laravel o CodeIgniter.
-

3. El Camino Exacto de Construcción (De 0 a 100)

Para no perderte ni frustrarte, debes seguir **estrictamente este orden secuencial**:

[PASO 1] → Configuración y Conexión PDO (configuracion/ + modelos/Conexion.php)

```

      | (Comprobar que conecta a MySQL)
      ▼
[ PASO 2 ] → Ayudantes de Seguridad (ayudantes/Validador.php y Sesion.php)
      | (Tener listas las funciones de sanitización y sesión)
      ▼
[ PASO 3 ] → Tu Primer Modelo: Usuario.php (Capa de Datos)
      | (Escribir y probar consultas SELECT e INSERT con PDO)
      ▼
[ PASO 4 ] → Tu Primer Controlador: UsuarioControlador.php (Lógica de Flujo)
      | (Recibir datos, validar con Validador.php y llamar a
Usuario.php)
      ▼
[ PASO 5 ] → Ensamblar la Vista: plantilla/ + usuarios/
      | (Inyectar datos dinámicos con PHP en tu maquetado HTML)
      ▼
[ PASO 6 ] → Prueba en el Navegador con index.php

```

4. Guía Detallada Paso a Paso

PASO 1: Conexión y Configuración

Objetivo: Asegurarte de que PHP puede hablar con tu base de datos MySQL.

1. Abre [codigo_fuente/configuracion/base_datos.php]
(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/configuracion/base_datos.php) y verifica que el nombre de la base de datos (**ascend**), usuario (**root**) y contraseña coincidan con tu servidor local (XAMPP/WAMP o Docker).
2. Abre [codigo_fuente/modelos/Conexion.php]
(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/modelos/Conexion.php).
3. **¿Cómo saber si funciona?:** Crea un archivo temporal en **publico/test_db.php**:

```

<?php
require_once '../modelos/Conexion.php';
$db = Conexion::obtenerConexion();
echo "¡Conexión a MySQL exitosa!";

```

Si entras desde el navegador a http://localhost/SGDM/codigo_fuente/publico/test_db.php y ves el mensaje, **tus cimientos están listos**.

PASO 2: Los Ayudantes de Seguridad y Sesión

Objetivo: Tener herramientas para no repetir código de validación.

1. Revisa [codigo_fuente/ayudantes/Validador.php]
(file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/ayudantes/Validador.php).
 - Aprende a usar **Validador::sanitizar(\$dato)** para limpiar textos contra XSS.

- Aprende a usar `Validador::validarEmail($email)` y `Validador::validarTelefono($tel)`.
 - 2. Revisa [codigo_fuente/ayudantes/Sesion.php] (file:///c:/Users/juani/Documents/GitHub/SGDM/codigo_fuente/ayudantes/Sesion.php).
 - Este archivo se encargará de que nadie entre al panel de administración sin estar logueado.
-

🔗 PASO 3: Programar tu Primer Modelo (`modelos/Usuario.php`)

Objetivo: Aprender a consultar e insertar datos en MySQL usando sentencias preparadas con PDO.

1. Estructura mental de un Modelo:

- En el constructor (`__construct`), guardas la conexión: `$this->db = Conexion::obtenerConexion();`.
- En cada método, preparas la consulta (`prepare`), ejecutas (`execute`) y retornas (`fetch` o `fetchAll`).

2. Ejemplo pedagógico para listar usuarios:

```
public function obtenerTodos(): array {
    $sql = "SELECT u.id, u.email, u.nombre_completo, u.telefono, r.nombre AS rol_nombre
           FROM usuarios u
           INNER JOIN roles r ON u.rol_id = r.id";
    $stmt = $this->db->query($sql);
    return $stmt->fetchAll(); // Devuelve una lista de arreglos
}
```

3. Ejemplo pedagógico para insertar con seguridad anti-inyección SQL:

```
public function registrar(array $datos): int {
    // 1. Encriptar la contraseña con BCrypt (exigencia de Ciberseguridad)
    $hash = password_hash($datos['contrasena'], PASSWORD_BCRYPT);

    // 2. Consulta con marcadores :email, :hash, etc.
    $sql = "INSERT INTO usuarios (email, contrasena_hash, nombre_completo, rol_id)
           VALUES (:email, :hash, :nombre, :rol_id)";

    $stmt = $this->db->prepare($sql);
    $stmt->execute([
        ':email' => $datos['email'],
        ':hash' => $hash,
        ':nombre' => $datos['nombre_completo'],
        ':rol_id' => $datos['rol_id']
    ]);

    return (int) $this->db->lastInsertId();
}
```

🔗 PASO 4: Programar tu Primer Controlador ([controladores/UsuarioControlador.php](#))

Objetivo: Controlar la lógica de negocio sin mezclar SQL ni HTML.

El Controlador sigue siempre este patrón de 4 pasos:

1. **Paso A:** ¿Tiene permiso el usuario? ([Sesion::requerirAdmin\(\)](#));).
2. **Paso B:** Si es una petición POST, valida los datos con [Validador.php](#).
3. **Paso C:** Llama al método del Modelo ([\\$this->usuarioModelo->registrar\(\\$datos\)](#)).
4. **Paso D:** Redirige al listado ([header\('Location: index.php?c=usuario&a=index'\)](#)) o carga la vista.

🔗 PASO 5: Transformar tu HTML en una Vista Dinámica PHP

Objetivo: Reemplazar los datos estáticos de prueba (*hardcodeados*) por las variables reales de MySQL.

1. Abre tu archivo de maquetación y cambia su extensión a [.php](#) (o inclúyelo desde tu controlador).
2. Usa la estructura modular con tu plantilla:

```
<?php
$titlePagina = "Usuarios - ASCEND";
require_once RUTA_VISTAS . 'admin/plantilla/cabecera.php';
require_once RUTA_VISTAS . 'admin/plantilla/menu_lateral.php';
?>

<h2>Listado de Usuarios</h2>
<table class="tabla-admin">
    <thead>
        <tr>
            <th>ID</th>
            <th>Nombre</th>
            <th>Correo</th>
            <th>Rol</th>
        </tr>
    </thead>
    <tbody>
        <?php foreach ($usuarios as $u): ?>
            <tr>
                <td><?= htmlspecialchars($u['id']) ?></td>
                <td><?= htmlspecialchars($u['nombre_completo']) ?></td>
                <td><?= htmlspecialchars($u['email']) ?></td>
                <td><span class="badge"><?= htmlspecialchars($u['rol_nombre']) ?
            </span></td>
            </tr>
        <?php endforeach; ?>
    </tbody>
</table>

<?php
require_once RUTA_VISTAS . 'admin/plantilla/pie_pagina.php';
?>
```

🔍 5. Herramientas y Trucos de Depuración (Debugging) para Principiantes

Cuando un código no funcione o una variable esté vacía, utiliza estas 3 herramientas nativas de PHP:

1. **`var_dump($variable); die();`**: Imprime en pantalla el contenido exacto, tipo de dato y estructura de cualquier variable y detiene la ejecución inmediatamente.

```
$usuarios = $this->usuarioModelo->obtenerTodos();  
var_dump($usuarios); // Muestra todo lo que trajo la base de datos  
die();
```

2. **Ver errores en pantalla durante el desarrollo**: Al inicio de `publico/index.php` puedes activar:

```
ini_set('display_errors', 1);  
ini_set('display_startup_errors', 1);  
error_reporting(E_ALL);
```

3. **Probar URLs directamente en el navegador**:

- Para ver el listado de usuarios:
`http://localhost/SGDM/codigo_fuente/publico/index.php?c=usuario&a=index`
- Para abrir el formulario de crear:
`http://localhost/SGDM/codigo_fuente/publico/index.php?c=usuario&a=crear`
- Para ir al dashboard: `http://localhost/SGDM/codigo_fuente/publico/index.php?c=admin&a=dashboard`

🕒 6. Tu Plan de Trabajo Personal como Programador del Admin

Para la próxima semana, tu meta debe ser:

1. **Día 1**: Verificar la conexión PDO y levantar el `index.php` en el navegador.
2. **Día 2**: Programar `Usuario.php` (métodos `obtenerTodos` y `registrar`).
3. **Día 3**: Programar `UsuarioControlador.php` y conectar la vista con la tabla de usuarios.
4. **Día 4**: Programar el alta de nuevos usuarios desde el formulario.
5. **Día 5**: Replicar la misma lógica para el catálogo de **Disciplinas / Juegos** (`Juego.php` y `JuegoControlador.php`).

Siguiendo esta guía, en menos de una semana tendrás el panel administrativo funcionando al 100% en PHP con arquitectura profesional.